

Formal Languages and Compilers

(Linguaggi Formali e Compilatori)

prof. Luca Breveglieri

(prof. A. Morzenti)

Written exam - 2 July 2014 - Part I: Theory

NAME:

MATRICOLA:

SIGNATURE:

INSTRUCTIONS - READ CAREFULLY:

- The exam consists of two parts:
 - I (80%) Theory:
 1. regular expressions and finite automata
 2. free grammars and pushdown automata
 3. syntax analysis and parsing
 4. translation and semantic analysis
 - II (20%) Practice on Flex and Bison
- To pass the exam, the candidate must succeed in both parts (I and II), in one call or more calls separately, but within one year.
- To pass part I (theory) one must answer the mandatory (not optional) questions. Notice the full grade is achieved by answering the optional questions.
- The exam is open book (texts and personal notes are admitted).
- Please write in the free space left and if necessary continue on the back side of the sheet; do not attach new sheets nor replace the existing ones.
- Time: Part I (theory): 2h.15m - Part II (practice): 60m

1 Regular Expressions and Finite Automata 20%

1. Consider the following unilinear grammar G (axiom S):

$$G \left\{ \begin{array}{l} S \rightarrow a T \\ T \rightarrow a \mid b T \mid a W \\ W \rightarrow \varepsilon \mid a T \end{array} \right.$$

Answer the following questions:

- (a) By using the systematic method that produces a transition for each rule, transform grammar G into an automaton A , possibly nondeterministic.
 - (b) If the automaton A found before is deterministic, construct an equivalent deterministic automaton A' , by means of a systematic method of your choice.
 - (c) By using the method of the linear language equations, from grammar G obtain a regular expression R that generates language $L(G)$.
 - (d) (optional) By using the Berry-Sethi method, from the regular expression R found before obtain a deterministic automaton A'' that recognizes language $L(R)$.
-

2 Free Grammars and Pushdown Automata 20%

1. Consider the so-called *dangling else* case found in some programming languages. A way to solve it is to associate the dangling `else` branch to the *outermost* if conditional that can bear it. Here is an example, below on the left:

```
if expression then
    if expression then
        assignment
else
    assignment
```

example for question (a)

```
if expression then
    assignment;
    if expression then
        assignment;
        assignment
else
    if expression then
        assignment
else
    assignment
```

example for question (b)

The dangling `else` branch is associated as shown by the alignment, that is to the outer if conditional. Notice this way of solving the dangling `else` case is quite different from the one usually adopted in most programming languages (e.g., the *C* language).

The branches of the if conditional contain only one statement: an assignment or a nested if conditional, and so on recursively. The expression and the assignment are generated by nonterminals *E* and *A*, resp., which do not need to be expanded.

Answer the following questions:

- (a) Write a *BNF* grammar *G*, *not ambiguous*, that generates the if conditional (or simply an assignment) and solves the dangling `else` case as explained before.
- (b) (optional) Extend the language and let the branches of the if conditional contain a non-empty block of assignments and nested conditionals. Every statement in a block is followed by a semicolon ‘;’, except the last statement. See the example above on the right, where the last `else` is associated to the outermost if.

Write a *BNF* grammar *G'*, *of any type*, that generates the extended language and solves the dangling `else` case as explained before. Generating a block of statements may require additional nonterminals, to be expanded.

2. One wishes one modeled a syntactic construct for defining a new data type: a *record with variant field*. Such a new record type looks like a standard record, e.g., a *struct* of the *C* language, which in addition may have one (or more) field(s) that can change name and type according to the value of a *discriminant* tag. Such a tag must be of enumeration type, declared separately.

In the example below, type `dataDescriptor` has two variants, according to the values of the discriminant tag `kind` of type enumeration `addressType`: in the variant with discriminant value `absolute`, the record field is called `absAddr` and is a natural number; instead, in the variant with discriminant value `offset` the record field is called `offAddr` and is an integer.

Consider the following restrictions:

- the declarations of enumeration type must list at least two possible values (labels)
- type declarations may be nested, meaning the type of a record field may be specified by an identifier (declared somewhere else in the program) or by a full type declaration (possibly of record type) extensively written in place
- although nested record declarations are permitted, it is forbidden to nest the *case* clauses; in other words, a variant field in a record may not in turn be of type record with variant field

Here is an example, with an enumeration type and a type record with variant field:

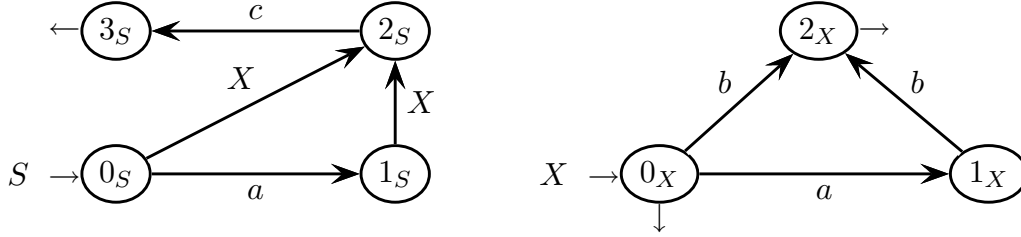
```
type addressType = enum absolute, offset endEnum;  
type dataDescriptor = record  
  size: natural;  
  case kind: addressType of  
    if absolute then absAddr: natural;  
    if offset then offAddr: integer;  
  endCase;  
endRecord
```

The language consists of a non-empty list of declarations of enumeration type and of type record with variant field, as explained before.

Write an *EBNF* grammar, not ambiguous, that generates the language fragment described above.

3 Syntax Analysis and Parsing 20%

1. Consider the following recursive net of machines $\mathcal{M}$ (axiom S):



Answer the following questions:

- Draw the pilot of net $\mathcal{M}$, say if the net is of type $ELR(1)$, and if it is not, then list all the conflicts in the net.
- Net $\mathcal{M}$ is not of type $ELL(1)$. Draw the *PCFG* (parser control-flow graph) of net $\mathcal{M}$, show on it all the relevant guide sets and indicate where the net is not of type $ELL(1)$. Then try to find a new net $\mathcal{M}'$, equivalent to net $\mathcal{M}$, but which is of type $ELL(1)$.
- (optional) By means of the Earley algorithm, analyze the string below:

abc

which belongs to language $L(\mathcal{M})$. Draw the syntax tree (or all the trees) of this string. Please use the table prepared on the next page.

Earley vector of string $a b c$ (to be filled)
 (the number of rows is not significant)

0	a	1	b	2	c	3

syntax tree(s) of string $a b c$ (to be drawn)

4 Translation and Semantic Analysis 20%

1. In a hypothetical programming language, one wishes one transformed the iterative statements of the following type (cycle with initial condition):

```
do
    a = a - 1;
end if (a = 0);
```

in three possible ways, shown by the three sample translations below:

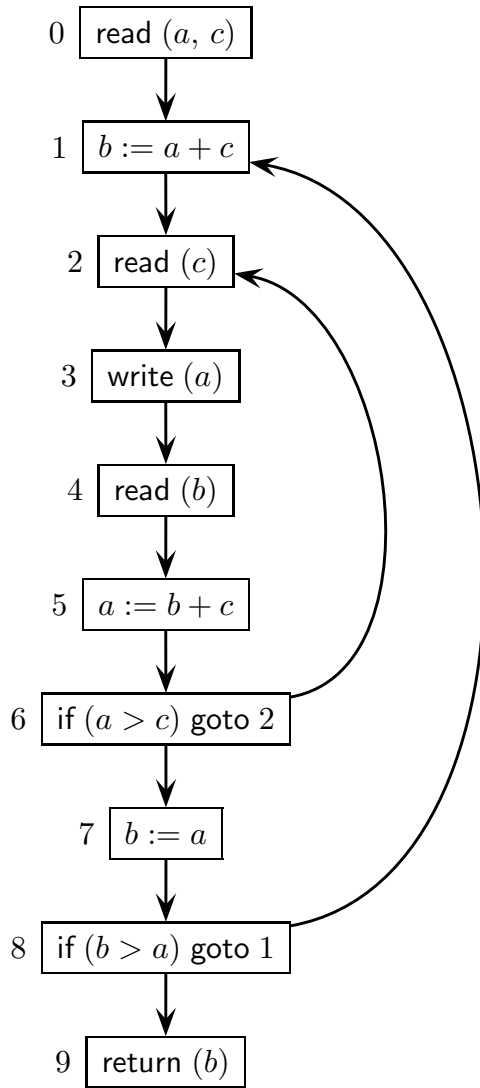
- (i) repeat
 a = a - 1;
 until (a = 0);
- (ii) repeat
 a = a - 1;
 while ! (a = 0);
- (iii) while ! (a = 0)
 a = a - 1;
 end while;

Notice the symbol ‘!’ indicates the logical negation operator, as it usually happens in many programming languages. The body of the cycle is a generic block of statements, including possible nested iterative statements.

Answer the following questions:

- (a) For each of the three ways shown above, say if the translation is syntactic and in such a case write its syntactic translation scheme (or its translation grammar), or say if it is not syntactic and explain why.
 - (b) (optional) If the translation is syntactic, say if it is computable by a finite state translator that scans the program one way, or possibly under what conditions such a finite state translation is computable.
-

2. Consider the following control-flow graph of a program (initial node 0) with three integer variables a , b and c , and an output parameter:



node	use
0	
1	
2	
3	
4	
5	
6	
7	
8	
9	

node	def
0	
1	
2	
3	
4	
5	
6	
7	
8	
9	

Answer the following questions:

- Write a regular expression R (over the alphabet $[0 - 9]$) that represents the set of program execution traces, without considering the instruction semantics. Then say if such an expression R is representative also when considering the instruction semantics, and possibly refine it.
- By means of the flow equation method, find the *live variables* at the program nodes and write them on the control-flow graph by the node boxes. Please use also the tables prepared aside the program graph and those on the next page.
- (optional) Say which definitions of variable reach (*reaching definitions*) the program nodes number 9 (output node) and number 3, and explain why.

system of data-flow equations for live variables

<i>node</i>	<i>in equations</i>	<i>out equations</i>
0		
1		
2		
3		
4		
5		
6		
7		
8		
9		

iterative solution table of the system of data-flow equations

(the number of columns is not significant)

#	<i>out</i>	<i>in</i>	<i>out</i>	<i>in</i>	<i>out</i>	<i>in</i>	<i>out</i>	<i>in</i>	<i>out</i>	<i>in</i>
0										
1										
2										
3										
4										
5										
6										
7										
8										
9										

